

目 录

代码评审

Phabricator

代码评审工具Phabricator

phabricator代码评审使用手册

流程图

代码评审

前言

代码评审也叫CodeReview，主要目的如下：

提高代码质量：代码审查可以帮助发现代码中的问题和缺陷，比如代码逻辑错误、安全漏洞、代码注释不足等，及时修改这些问题和缺陷可以提高代码的质量和可靠性，减少后期修复和维护的时间和成本。

减少错误率和缺陷率：通过代码审查可以发现并纠正代码中的错误、缺陷和不规范的代码，从而减少代码中的错误率和缺陷率，提高软件的质量和稳定性。

提高开发效率：代码审查可以帮助团队成员相互学习和分享经验，提高开发人员的技能和能力，同时也可以减少代码的重复和冗余，提高开发效率。

促进团队协作和交流：代码审查可以促进团队成员之间的协作和交流，增加团队合作的感觉，从而提高团队的凝聚力和工作效率。

提高安全性：代码审查可以帮助发现代码中的安全漏洞和风险，及时加以修复和改进，从而提高软件的安全性和防御能力，保护用户的隐私和数据安全。

Phabricator

前言

代码评审也叫CodeReview，主要目的如下：

提高代码质量：代码审查可以帮助发现代码中的问题和缺陷，比如代码逻辑错误、安全漏洞、代码注释不足等，及时修改这些问题和缺陷可以提高代码的质量和可靠性，减少后期修复和维护的时间和成本。

减少错误率和缺陷率：通过代码审查可以发现并纠正代码中的错误、缺陷和不规范的代码，从而减少代码中的错误率和缺陷率，提高软件的质量和稳定性。

提高开发效率：代码审查可以帮助团队成员相互学习和分享经验，提高开发人员的技能和能力，同时也可以减少代码的重复和冗余，提高开发效率。

促进团队协作和交流：代码审查可以促进团队成员之间的协作和交流，增加团队合作的感觉，从而提高团队的凝聚力和工作效率。

提高安全性：代码审查可以帮助发现代码中的安全漏洞和风险，及时加以修复和改进，从而提高软件的安全性和防御能力，保护用户的隐私和数据安全。

代码评审工具Phabricator

一、Phabricator是什么？

- 1、在代码审查(Code Review)方面，Facebook做了一个可视化的工具，现已开源，叫Phabricator。
- 2、负责审查的工程师可以接受代码改变，可以提出疑问要求原作者继续修改，可以提出自己不适合以推出该代码审查。
- 3、工程师可以在页面上非常方便的针对每一段(单行或者多行)代码进行交互讨论。

二、Phabricator的优点

- 1、不光只有代码review工具，还有bug跟踪，wiki等功能，可以直接做单元测试,bug与代码review的关联。
- 2、按request状态分类清晰，搜索功能好用。
- 3、支持svn 和 git。
- 4、所有检视工作只需要一个浏览器，不需要安装额外的插/软件。
- 5、操作界面和易用性非常棒。

三、Phabricator的常见用途

- 1、代码仓库
- 2、代码审查

四、使用 Phabricator评审代码的通俗实例

- 1、A进行了一次代码提交
- 2、B接收到审核请求
- 3、过了一阵儿，B登录Phabricator并在首页看到审核请求
- 4、B检查A提交的代码。他发现代码中的一些问题，之后他选择了“请求变更”选项，并且在评论中描述了这些问题
- 5、A收到一封关于B对她的请求变更的email。她决定过一会儿再处理这个问题
- 6、不久后，B登录Phabricator并在首页“问题提交”下看到提示
- 7、A通过某些方式解决了那些问题（如“找B讨论”、“修复问题并提交”）
- 8、B表示满意，并认可了最初那次提交
- 9、审核请求将从B的待办事项中消失。问题提交也会从A的待办事项里消失

五、Phabricator使用的几点说明

- 1、责任感。在审阅一次代码提交时，你所负责的审核是被突出显示的。你要为自己的任何审核行为负责。
- 2、在diff对比区域，点击行号将可添加内嵌评论。
- 3、在diff对比区域，在行号上拖动可添加跨越多行的内嵌评论。
- 4、内嵌评论最初只保存为草稿，直到你在页面底部提交评论。
- 5、按“?”键查看快捷键。

phabricator代码评审使用手册

一、初始化环境

1、设置默认arc编辑器为vim，设置url

```
arc set-config editor "vim"  
arc set-config default "http://172.16.100.95:9999"
```

生成的配置文件路径为 ~/.arcrc

2、安装证书

```
ps@hichip01:~/work/project01$ arc install-certificate  
CONNECT Connecting to "http://172.16.100.95:9999/api/..."  
LOGIN TO PHABRICATOR  
Open this page in your browser and login to Phabricator if necessary:  
  
http://172.16.100.95:9999/conduit/login/  
  
Then paste the API Token on that page below.  
  
Paste API Token from that page: cli-ydlxr43lxzpn2j2hymccrfntcxx
```

备注：在浏览器输入<http://172.16.100.95:9999/conduit/login/>，输入phabricator的用户和密码获取API Token

二、代码提交评审流程（开发者）

1、进行开发活动

进入项目创建或修改代码

```
cd project_path  
  
git add .  
  
git commit -m "add 注释06"
```

2、开发者提交code review审核流程

输入git commit -m "描述"命令后即可提交代码评审流程

```
命令：arc diff --create
```

在代码目录中，执行以下命令：

```
ps@hichip01:~/work/project01$ arc diff --create
```

这将根据本地的commit创建一个新的Diff，并自动打开一个编辑器以供填写Diff的详细说明。

编辑器中包含了以下字段：

标题：简洁明了地描述更改的概要（此处标题会引用git commit 的描述信息）。

摘要：提供更详细的描述，解释的更改的原因、目的和影响等。

测试计划：说明如何测试的更改，包括所需的环境、步骤和预期的结果（如没有测试计划，请填写任意字符,比如输入no）。

评审人：填写代码评审人,如有多个评审人，需要用“,”号隔开

订阅人：填写抄送人（选填），,如有多个订阅人，需要用“,”号隔开

填写完毕后，保存并关闭编辑器。arc diff命令将会提交审核流程到Phabricator。

示例：

```
add 注释06

摘要:增加了日期注释

测试计划: no

评审人: root

订阅者:

# NEW DIFFERENTIAL REVISION
# Describe the changes in this new revision.
#
# Included commits in branch master:
#
#       93762844e164 add 注释06
#
# arc could not identify any existing revision in your working copy.
# If you intended to update an existing revision, use:
#
#   $ arc diff --update <revision>
~
~
~
~
~
-- INSERT --
```

也可以跳过编辑器直接提供说明信息

默认情况下，`arc diff` 命令会打开一个编辑器供填写Diff的说明信息。如果希望通过命令行参数直接提供说明信息，可以使用 `--verbatim` 选项。执行以下命令：

```
arc diff --verbatim --title "My Diff Title" --summary "My Diff Summary" --test-plan "My Test Plan" --reviewers "user"
```

可以使用 `--title`、`--summary` 和 `--test-plan` 参数分别指定标题、摘要和测试计划的内容。

其他选项和高级功能

除了上述功能外，arc diff命令还提供了其他选项和高级功能，以满足不同的审查需求。以下是一些常用选项的示例：

```
--allow-untracked：允许提交未跟踪的文件。  
--reviewers <reviewers>：指定审核人。  
--cc <cc>：指定抄送人。  
--base <commit>：指定基准提交。  
--skip-staging：跳过分期区域，直接提交Diff。
```

三、代码审核流程（审核者）

1、审核人进行代码审核

普通开发者提交代码评审流程后，审核人会收到一封邮件

[Differential] **[请求]** **[+]** **D14: add 注释06** ★

bill.huang (黄庭彪)
发给 it

发件人: bill.huang (黄庭彪) <jenkins@hichiptech.com>
收件人: it <it@hichiptech.com>
时间: 2023年8月15日 (周二) 10:10
大小: 5 KB

bill.huang 创建了此修订。
bill.huang 添加了 1 位评审人: root。
bill.huang 请求了此修订的评审。

修订摘要
增加了日期注释

测试计划
no

仓库
rPROJECTXX Project01

分支
master

修订详情
<http://172.16.100.95:9999/D14>

受影响的文件
config/dev.env.js

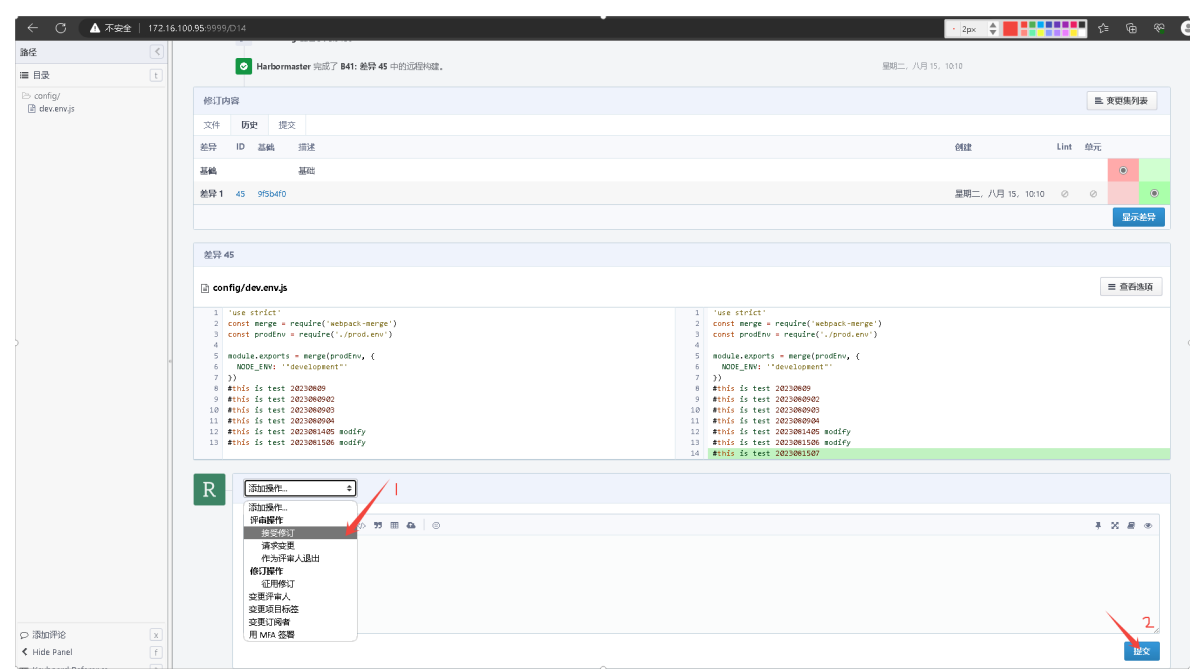
邮件偏好设置
<http://172.16.100.95:9999/settings/panel/emailpreferences/>

To: bill.huang, root

审核人点击修订详情的URL即可进入审核页面

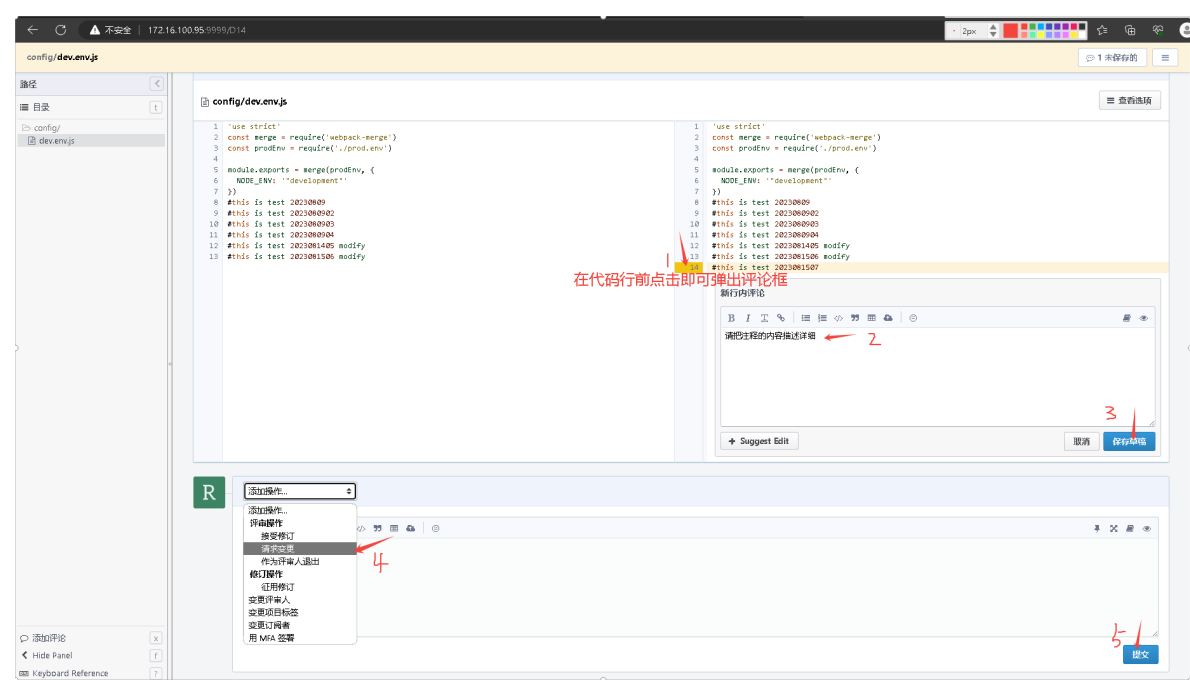
情景一：代码没问题，可以推送合并到远程代码仓库

在评论框左上的菜单中选择“接受修订”后提交即可。



情景二：代码不符合要求，需要修改代码

在评论框中添加评论，选择“请求变更”选项后点击提交，开发者将会收到一封请求变更的邮件。



四、代码审核结果（开发者）

1、已提交code review，等待审核人进行审核

情景一：审核通过

注意查看邮件，邮件显示标题[Differential] [已接受] D13：，代表审核通过，按照正常的git 流程把代码推送到远端的gitlab仓库



例如：git push origin master

情景二：审核不通过

如收到邮件显示[Differential] [请求的变更] D13：，即需要按照审核者的评论意见修改你的代码，修改后再进行code review流程，等待审核，审核通过后按照正常的git 流程把代码推送到远端的gitlab仓库

[Differential] [请求的变更] D13: add 注释 ★

root (root)
发给 bill.huang

发件人: root (root) <jenkins@hichiptech.com>
收件人: bill.huang <bill.huang@hichiptech.com>
时间: 2023年8月15日 (周二) 09:31
大小: 7 KB

root 请求了对此修订的变更。
root 添加了行内评论。
此修订现在需要处理变更。

行内评论

dev.env.js:13

```
#this is test 2023081405 modify  
#this is test 2023081506
```

请把注释的内容描述详细

仓库
rPROJECTXX Project01

自上次操作以来的变更
<http://172.16.100.95:9999/D13/new/>

修订详情
<http://172.16.100.95:9999/D13>

邮件偏好设置
<http://172.16.100.95:9999/settings/panel/emailpreferences/>

To: bill.huang, root

```
cd /work/project01  
修改代码。。。。。  
git add .  
git commit -m "modify dev.env.js"  
arc diff --update D13
```

[illegible]

备注：在弹出的编辑器填写主题和注释，第一行为主题（默认会读取git commit中的描述作为主题），第二行填写摘要。

2、审核通过后push后自动关闭该评审会话

如需手动关闭，使用以下命令

查看会话的revision

命令： `arc list`

```
ps@hichip01:~/work/project01/config$ arc list
* 已接受 D13: modify prod.env.js 02
```

arc close-revision D13

五、arc常用命令

1、arc的一些命令用法：

arc 包含很多子命令，

arc help 列出来子命令看看

arc diff 调用svn diff或git diff生成差异并发送给phabricator生成评审单

arc branch 查看分支的审核情况

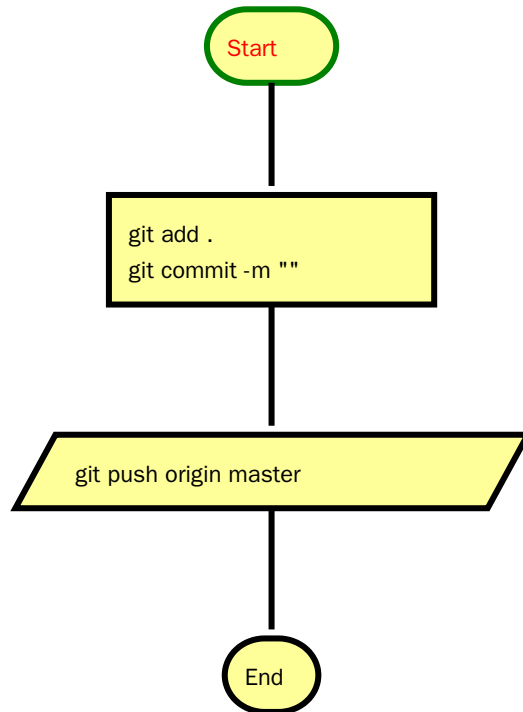
arc list 列出当前peding的revision —— revision要说明一下：pha生产的每个评审单都对应一个revision，可以理解为svn/git的一次提交，但又不在用户的svn/git上体现，可以理解为pha上也驻留这一个svn/git，来存储评审单信息，每单对应一个revision。

arc patch 将pha上的revision变更patch到本地工作拷贝上

arc amend 更新git commit的message，即：可以把pha上某个revision的message应用到本地git的某个commit上
arc land git专用，pha上评审完毕后，将本地分支做git push，所以需要有 origin
arc unit 执行单元测试，这个就需要用户自己来指定单元测试工具了
arc close-revision 使用arc关闭某个revision，而不必上pha上鼠标点点点啦

流程图

原来的提交流程：



审核机制提交流程：

